

実験 第3回レポート

学籍番号 2022311042 名前 神野友哉
2023年10月24日

1 実験

1.1 目的

1.2 方法

1.3 結果

1.4 コメント

2 実験 3-1

実験 3-1 に用いたラベリング手法の原理を説明し、実験結果が正しい裏付けになる出力画像 G のデータの一部を示せ

2.1 考察



図 1: zukei.pbm

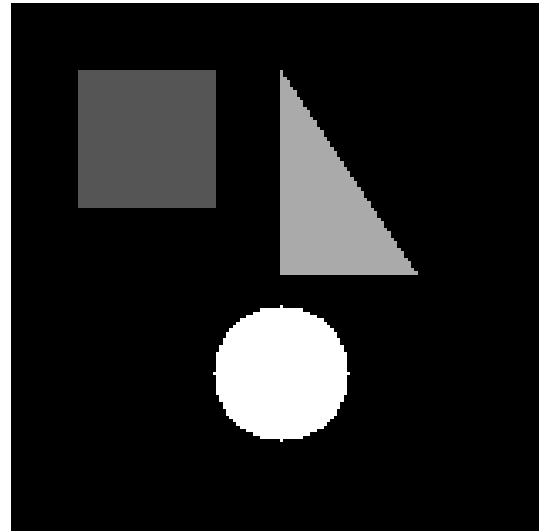


図 2: ラベリング後

ソースコード 1: rinkaku-label-pbm.c

```

1 // pbm ファイルから画像データの読込とファイルへの書き出し
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6
7 #define DIRECTION_MAX_INTEGER 8
8
9 int main(void)
10 {
11     FILE *fpi = NULL; // 入力画像
12     FILE *fpo = NULL; // 出力画像
13
14     char buf[258];
15     int width, height; // 画像の横長と縦長
16     int maxvalue; // 画像の最大輝度値
17     int i, x, y;
18     char fnamein[50];
19     char fnamein_buf[50];
20     char fnameout[50];
21     char fnameout_right[10];
22     char *tok;
23     int pro;
24
25     puts("what name is your file? Example name.pbm");
26     scanf("%s", fnamein);
27     for (i = 0; i < sizeof(fnamein_buf); i++) {
28         fnamein_buf[i] = fnamein[i];
29     }

```

```

30 tok = strtok(fnamein_buf, ".");
31 sprintf(fnameout, "%s-label-out.pgm", tok);
32
33 // ファイル fnamein から画像データを読み込む
34
35 fpi = fopen(fnamein, "r");
36
37 if (fpi == NULL) printf("Reading file open error ! %s\n", fnamein);
38
39 do {
40     fgets(buf, 256, fpi);
41 } while (buf[0] == '#'); // skip over comments
42
43 if (buf[0] != 'P' || buf[1] != '1') {
44     fclose(fpi);
45     printf(" \n The input image is not pbm format! \n\n");
46     return -1;
47 }
48
49 do {
50     fgets(buf, 256, fpi);
51 } while (buf[0] == '#'); // skip over comments
52
53 sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
54
55 int **img;
56 int **bufArr;
57 img = (int **)calloc(height, sizeof(int *));
58 bufArr = (int **)calloc(height, sizeof(int *));
59 for (i = 0; i < height; i++) {
60     img[i] = (int *)calloc(width, sizeof(int));
61     bufArr[i] = (int *)calloc(width, sizeof(int));
62 }
63 if (img == NULL || bufArr == NULL) {
64     printf("failed to get enough of memory\n");
65     return -1;
66 }
67
68 for (y = 0; y < height; y++) {
69     for (x = 0; x < width; x++) {
70         fscanf(fpi, "%d", &img[y][x]);
71         bufArr[y][x] = 0;
72     }
73 } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
74
75 fclose(fpi);
76
77 int dir_current = 6;
78 int dir_next = 4;
79 int skip_frag;

```

```

80  int x_c, y_c;
81  int p_c = 0; // 1 黒 0 白
82  int label = 0;
83  for (y = 0; y < height; y++) {
84      for (x = 0; x < width; x++) {
85          if (img[y][x] && !bufArr[y][x] && !p_c) {
86              bufArr[y][x] = 1;
87              x_c = x;
88              y_c = y;
89              dir_current = 6;
90              label++;
91              do {
92                  skip_frag = 0;
93                  dir_next = (dir_current - 2 + DIRECTION_MAX_INTEGER) %
                      DIRECTION_MAX_INTEGER;
94                  for (i = 0; i < DIRECTION_MAX_INTEGER; i++) {
95                      if (i)
96                          dir_next = (dir_next + 1 + DIRECTION_MAX_INTEGER) %
                              DIRECTION_MAX_INTEGER;
97                      switch (dir_next) {
98                          case 0:
99                          if (x_c < width - 1 && img[y_c + 0][x_c + 1]) {
100                              x_c++;
101                              skip_frag++;
102                          }
103                          break;
104                          case 1:
105                          if (x_c < width - 1 && y_c > 0 && img[y_c - 1][x_c + 1]) {
106                              x_c++;
107                              y_c--;
108                              skip_frag++;
109                          }
110                          break;
111                          case 2:
112                          if (y_c > 0 && img[y_c - 1][x_c + 0]) {
113                              y_c--;
114                              skip_frag++;
115                          }
116                          break;
117                          case 3:
118                          if (x_c > 0 && y_c > 0 && img[y_c - 1][x_c - 1]) {
119                              x_c--;
120                              y_c--;
121                              skip_frag++;
122                          }
123                          break;
124                          case 4:
125                          if (x_c > 0 && img[y_c + 0][x_c - 1]) {
126                              x_c--;
127                              skip_frag++;

```

```

128         }
129         break;
130     case 5:
131         if (x_c > 0 && y_c < height - 1 && img[y_c + 1][x_c - 1]) {
132             x_c--;
133             y_c++;
134             skip_frag++;
135         }
136         break;
137     case 6:
138         if (y_c < height - 1 && img[y_c + 1][x_c + 0]) {
139             y_c++;
140             skip_frag++;
141         }
142         break;
143     case 7:
144         if (x_c < width - 1 && y_c < height - 1 && img[y_c + 1][x_c +
145             1]) {
146                 x_c++;
147                 y_c++;
148                 skip_frag++;
149             }
150         break;
151     if (skip_frag) {
152         bufArr[y_c][x_c] = label;
153         dir_current = dir_next;
154         break;
155     }
156 }
157 } while (x_c != x || y_c != y);
158 }
159 p_c = img[y][x] ? 1 : 0;
160 }
161 }
162
163 for (y = 1; y < height; y++) {
164     for (x = 1; x < width; x++) {
165         if(!bufArr[y][x] && bufArr[y][x-1] && img[y][x]) bufArr[y][x] = bufArr
166             [y][x-1];
167         if(!bufArr[y][x] && bufArr[y-1][x] && img[y][x]) bufArr[y][x] = bufArr
168             [y-1][x];
169     }
170 }
171
172 maxvalue = 255;
173 for (y = 0; y < height; y++) {
174     for (x = 0; x < width; x++) {
175         img[y][x] = bufArr[y][x] * (maxvalue / label);
176     }
177 }

```

```

175 }
176
177 // 画像データをファイル fnameout に出力する
178 fpo = fopen(fnameout, "w");
179
180 if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
181
182 fprintf(fpo, "P2\n");
183 fprintf(fpo, "# %s\n", fnameout);
184 fprintf(fpo, "%d %d \n", width, height);
185 fprintf(fpo, "%d\n", maxvalue);
186
187 int count = 0;
188
189 for (y = 0; y < height; y++) {
190     for (x = 0; x < width; x++) {
191         fprintf(fpo, "%1d ", img[y][x]);
192         count++;
193         if (count % 18 == 0) {
194             fprintf(fpo, "\n"); // 1行 18個画素
195         }
196     }
197 }
198 fclose(fpo); // 出力画像をクローズする
199
200 for (i = 0; i < height; i++) {
201     free(img[i]);
202     free(bufArr[i]);
203 }
204 free(img);
205 free(bufArr);
206 }

```

3 実験 3-2

実験 3-2 に用いる特徴計算の方法を説明し、各物体の中心点、周囲長、面積と複雑度を表で示せ

3.1 考察



図 3: zukei.pbm

```
[is22i042@ie015 exp-3]$ gcc rinkaku-fvalue-pgm.c -lm -o rinkaku-fvalue-pgm && ./rinkaku-fvalue-pgm
what name is your file? Example name.pgm
zukei-label-out.pgm
label : 1, square : 1681, length : 160.00, cpoint : (40, 40), Cvalue : 0.83
label : 2, square : 1261, length : 176.57, cpoint : (93, 60), Cvalue : 0.51
label : 3, square : 1257, length : 131.88, cpoint : (80, 110), Cvalue : 0.91
[is22i042@ie015 exp-3]$
```

図 4: 特徴量

ソースコード 2: rinkaku-fvalue-pgm.c

```
1 // pbm ファイルから画像データの読込とファイルへの書き出し
2 #define _USE_MATH_DEFINES
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6 #include <math.h>
7
8 #define DIRECTION_MAX_INTEGER 8
9
10 double followOutlineAndGetLength(int** ori, int** buf, int width, int height,
    int* dc, int x, int y, int label) {
11     int skip_frag;
12     int dn;
13     int x_c = x;
```

```

14     int y_c = y;
15     double length = 0;
16     do {
17         skip_frag = 0;
18         dn = (*dc - 2 + DIRECTION_MAX_INTEGER) % DIRECTION_MAX_INTEGER;
19         for (int i = 0; i < DIRECTION_MAX_INTEGER; i++) {
20             if (i)
21                 dn = (dn + 1 + DIRECTION_MAX_INTEGER) % DIRECTION_MAX_INTEGER;
22             switch (dn) {
23                 case 0:
24                     if (x_c < width - 1 && ori[y_c + 0][x_c + 1]) {
25                         x_c++;
26                         skip_frag++;
27                     }
28                     break;
29                 case 1:
30                     if (x_c < width - 1 && y_c > 0 && ori[y_c - 1][x_c + 1]) {
31                         x_c++;
32                         y_c--;
33                         skip_frag++;
34                     }
35                     break;
36                 case 2:
37                     if (y_c > 0 && ori[y_c - 1][x_c + 0]) {
38                         y_c--;
39                         skip_frag++;
40                     }
41                     break;
42                 case 3:
43                     if (x_c > 0 && y_c > 0 && ori[y_c - 1][x_c - 1]) {
44                         x_c--;
45                         y_c--;
46                         skip_frag++;
47                     }
48                     break;
49                 case 4:
50                     if (x_c > 0 && ori[y_c + 0][x_c - 1]) {
51                         x_c--;
52                         skip_frag++;
53                     }
54                     break;
55                 case 5:
56                     if (x_c > 0 && y_c < height - 1 && ori[y_c + 1][x_c - 1]) {
57                         x_c--;
58                         y_c++;
59                         skip_frag++;
60                     }
61                     break;
62                 case 6:
63                     if (y_c < height - 1 && ori[y_c + 1][x_c + 0]) {

```

```

64         y_c++;
65         skip_frag++;
66     }
67     break;
68     case 7:
69         if (x_c < width - 1 && y_c < height - 1 && ori[y_c + 1][x_c +
70             1]) {
71             x_c++;
72             y_c++;
73             skip_frag++;
74         }
75         break;
76     }
77     if (skip_frag) {
78         buf[y_c][x_c] = label;
79         length += (dn % 2 == 0) ? 1 : sqrt(2);
80         *dc = dn;
81         break;
82     }
83     } while (x_c != x || y_c != y);
84     return length;
85 }
86
87 void fillInside(int** ori, int** buf, int width, int height) {
88     int x, y;
89     for (y = 1; y < height; y++) {
90         for (x = 1; x < width; x++) {
91             if(!buf[y][x] && buf[y][x-1] && ori[y][x]) buf[y][x] = buf[y][x
92                 -1];
93             if(!buf[y][x] && buf[y-1][x] && ori[y][x]) buf[y][x] = buf[y-1][x
94                 ];
95         }
96     }
97
98 int getnumTotal(int** buf, int label, int width, int height) {
99     int x, y;
100    int count = 0;
101    for (y = 0; y < height; y++) {
102        for (x = 0; x < width; x++) {
103            if(buf[y][x] == label) count++;
104        }
105    }
106    return count;
107 }
108
109 int getCPX(int** buf, int label, int width, int height) {
110     int x, y;
111     int sumx = 0;

```

```

111     for (y = 0; y < height; y++) {
112         for (x = 0; x < width; x++) {
113             if(buf[y][x] == label) sumx += x;
114         }
115     }
116     return sumx / getnumTotal(buf, label, width, height);
117 }
118
119 int getCPY(int** buf, int label, int width, int height) {
120     int x, y;
121     int sumy = 0;
122     for (y = 0; y < height; y++) {
123         for (x = 0; x < width; x++) {
124             if(buf[y][x] == label) sumy += y;
125         }
126     }
127     return sumy / getnumTotal(buf, label, width, height);
128 }
129
130 double calC(int numTotal, double length) {
131
132     return 4.0f * M_PI * numTotal / pow(length, 2.0f);
133 }
134
135 int main(void) {
136     FILE *fpi = NULL; // 入力画像
137     FILE *fpo = NULL; // 出力画像
138
139     char buf[258];
140     int width, height; // 画像の横長と縦長
141     int maxvalue; // 画像の最大輝度値
142     int i, x, y;
143     char fnamein[50];
144     char fnamein_buf[50];
145     char fnameout[50];
146     char fnameout_right[10];
147     char *tok;
148     int pro;
149
150     puts("what name is your file? Example name.pgm");
151     scanf("%s", fnamein);
152     for (i = 0; i < sizeof(fnamein_buf); i++) {
153         fnamein_buf[i] = fnamein[i];
154     }
155     tok = strtok(fnamein_buf, ".");
156     sprintf(fnameout, "%s-fvalue-out.pgm", tok);
157
158     // ファイル fnamein から画像データを読み込む
159
160     fpi = fopen(fnamein, "r");

```

```

161
162 if (fpi == NULL) printf("Reading file open error ! %s\n", fnamein);
163
164 do {
165     fgets(buf, 256, fpi);
166 } while (buf[0] == '#'); // skip over comments
167
168 if (buf[0] != 'P' || buf[1] != '2') {
169     fclose(fpi);
170     printf(" \n The input image is not pgm format! \n\n");
171     return -1;
172 }
173
174 do {
175     fgets(buf, 256, fpi);
176 } while (buf[0] == '#'); // skip over comments
177
178 sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
179 fscanf(fpi, "%d", &maxvalue);
180
181 int **img = (int **)calloc(height, sizeof(int *));
182 int **bufArr = (int **)calloc(height, sizeof(int *));
183 for (i = 0; i < height; i++) {
184     img[i] = (int *)calloc(width, sizeof(int));
185     bufArr[i] = (int *)calloc(width, sizeof(int));
186 }
187 if (img == NULL || bufArr == NULL) {
188     printf("failed to get enough of memory\n");
189     return -1;
190 }
191
192 for (y = 0; y < height; y++) {
193     for (x = 0; x < width; x++) {
194         fscanf(fpi, "%d", &img[y][x]);
195     }
196 } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
197
198 fclose(fpi);
199
200 int dir_current;
201 int p_c = 0; // 1 黒 0 白
202 int label = 1;
203 double length, C_value;
204 int total, cpx, cpy;
205 for (y = 0; y < height; y++) {
206     for (x = 0; x < width; x++) {
207         if (img[y][x] && !bufArr[y][x] && !p_c) {
208             bufArr[y][x] = 1;
209             dir_current = 6;

```

```

210         length = followOutlineAndGetLength(img, bufArr, width, height, &
                dir_current, x, y, label);
211         fillInside(img, bufArr, width, height);
212         total = getnumTotal(bufArr, label, width, height);
213         cpx = getCPX(bufArr, label, width, height);
214         cpy = getCPY(bufArr, label, width, height);
215         C_value = calC(total, length);
216         printf("label : %d, square : %d, length : %5.2f, cpoint : (%d, %d),
                Cvalue : %5.2f\n", label++, total, length, cpx, cpy, C_value);
217     }
218     p_c = img[y][x] ? 1 : 0;
219 }
220 }
221
222 maxvalue = 255;
223 for (y = 0; y < height; y++) {
224     for (x = 0; x < width; x++) {
225         img[y][x] = bufArr[y][x] * (maxvalue / (label - 1));
226     }
227 }
228
229 // 画像データをファイル fnameout に出力する
230 fpo = fopen(fnameout, "w");
231
232 if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
233
234 fprintf(fpo, "P2\n");
235 fprintf(fpo, "# %s\n", fnameout);
236 fprintf(fpo, "%d %d \n", width, height);
237 fprintf(fpo, "%d\n", maxvalue);
238
239 int count = 0;
240
241 for (y = 0; y < height; y++) {
242     for (x = 0; x < width; x++) {
243         fprintf(fpo, "%1d ", img[y][x]);
244         count++;
245         if (count % 18 == 0) {
246             fprintf(fpo, "\n"); // 1行 18個画素
247         }
248     }
249 }
250 fclose(fpo); // 出力画像をクローズする
251
252 for (i = 0; i < height; i++) {
253     free(img[i]);
254     free(bufArr[i]);
255 }
256 free(img);
257 free(bufArr);

```


4 実験 3-3

実験 3-3 の出力画像 G を示し、特徴計算の具体的な応用について考察せよ

4.1 考察



図 5: zukei.pbm

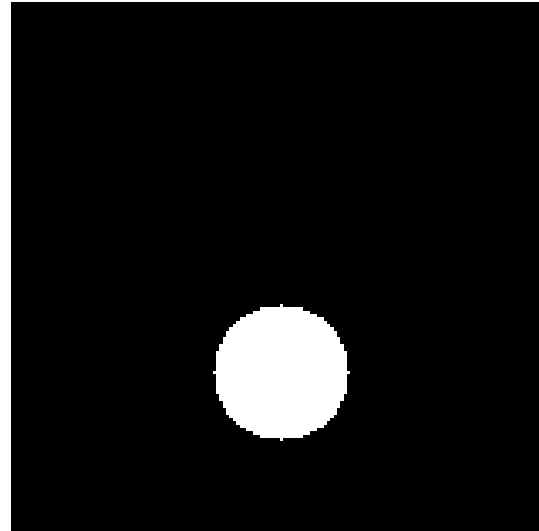


図 6: zukei-label-out-circle-out.pgm

ソースコード 3: rinkaku-circle-pgm.c

```

1 // pbm ファイルから画像データの読込とファイルへの書き出し
2 #define _USE_MATH_DEFINES
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6 #include <math.h>
7
8 #define DIRECTION_MAX_INTEGER 8
9
10 double followOutlineAndGetLength(int** ori, int** buf, int width, int height,
    int* dc, int x, int y, int label) {
11     int dn;
12     int x_c = x;
13     int y_c = y;
14     double length = 0;
15     int vec[8][6] = {{-1,width-1,-1,height,1,0},{-1,width-1,0,height
        ,1,-1},{-1,width,0,height,0,-1},{0,width,0,height,-1,-1},
16         {0,width,-1,height,-1,0},{0,width,-1,height-1,-1,1},{-1,
            width,-1,height-1,0,1},{-1,width-1,-1,height-1,1,1}};
17     do {
18         dn = (*dc - 2 + DIRECTION_MAX_INTEGER) % DIRECTION_MAX_INTEGER;
19         for (int i = 0; i < DIRECTION_MAX_INTEGER; i++) {
20             if(i) dn = (dn + 1 + DIRECTION_MAX_INTEGER) % DIRECTION_MAX_INTEGER;
21             if(x_c > vec[dn][0] && x_c < vec[dn][1] && y_c > vec[dn][2] && y_c
                < vec[dn][3] && ori[y_c + vec[dn][5]][x_c + vec[dn][4]]){
22                 x_c += vec[dn][4];
23                 y_c += vec[dn][5];
24                 buf[y_c][x_c] = label;
25                 length += (dn % 2 == 0) ? 1 : sqrt(2);
            }
        }
    } while (1);

```

```

26             *dc = dn;
27             break;
28         }
29     }
30 } while (x_c != x || y_c != y);
31 return length;
32 }
33
34 void fillInside(int** ori, int** buf, int width, int height) {
35     int x, y;
36     for (y = 1; y < height; y++) {
37         for (x = 1; x < width; x++) {
38             if(!buf[y][x] && buf[y][x-1] && ori[y][x]) buf[y][x] = buf[y][x
-1];
39             if(!buf[y][x] && buf[y-1][x] && ori[y][x]) buf[y][x] = buf[y-1][x
];
40         }
41     }
42 }
43
44 int getnumTotal(int** buf, int label, int width, int height) {
45     int x, y;
46     int count = 0;
47     for (y = 0; y < height; y++) {
48         for (x = 0; x < width; x++) {
49             if(buf[y][x] == label) count++;
50         }
51     }
52     return count;
53 }
54
55 int getCPX(int** buf, int label, int width, int height) {
56     int x, y;
57     int sumx = 0;
58     for (y = 0; y < height; y++) {
59         for (x = 0; x < width; x++) {
60             if(buf[y][x] == label) sumx += x;
61         }
62     }
63     return sumx / getnumTotal(buf, label, width, height);
64 }
65
66 int getCPY(int** buf, int label, int width, int height) {
67     int x, y;
68     int sumy = 0;
69     for (y = 0; y < height; y++) {
70         for (x = 0; x < width; x++) {
71             if(buf[y][x] == label) sumy += y;
72         }
73     }

```

```

74     return sumy / getnumTotal(buf, label, width, height);
75 }
76
77 double calC(int numTotal, double length) {
78
79     return 4.0f * M_PI * numTotal / pow(length, 2.0f);
80 }
81
82 void rmLabel(int** buf, int* label, int width, int height) {
83     int x, y;
84     for (y = 0; y < height; y++) {
85         for (x = 0; x < width; x++) {
86             if(buf[y][x] == *label) buf[y][x] = -1;
87         }
88     }
89     *label--;
90 }
91
92 int main(void) {
93     FILE *fpi = NULL; // 入力画像
94     FILE *fpo = NULL; // 出力画像
95
96     char buf[258];
97     int width, height; // 画像の横長と縦長
98     int maxvalue; // 画像の最大輝度値
99     int i, x, y;
100     char fnamein[50];
101     char fnamein_buf[50];
102     char fnameout[50];
103     char fnameout_right[10];
104     char *tok;
105     int pro;
106
107     puts("what name is your file? Example name.pgm");
108     scanf("%s", fnamein);
109     for (i = 0; i < sizeof(fnamein_buf); i++) {
110         fnamein_buf[i] = fnamein[i];
111     }
112     tok = strtok(fnamein_buf, ".");
113     sprintf(fnameout, "%s-circle-out.pgm", tok);
114
115     // ファイル fnamein から画像データを読み込む
116
117     fpi = fopen(fnamein, "r");
118
119     if (fpi == NULL) printf("Reading file open error ! %s\n", fnamein);
120
121     do {
122         fgets(buf, 256, fpi);
123     } while (buf[0] == '#'); // skip over comments

```

```

124
125 if (buf[0] != 'P' || buf[1] != '2') {
126     fclose(fpi);
127     printf(" \n The input image is not pgm format! \n\n");
128     return -1;
129 }
130
131 do {
132     fgets(buf, 256, fpi);
133 } while (buf[0] == '#'); // skip over comments
134
135 sscanf(buf, "%d %d", &width, &height); // 画像サイズを読み込む
136 fscanf(fpi, "%d", &maxvalue);
137
138 int **img = (int **)calloc(height, sizeof(int *));
139 int **bufArr = (int **)calloc(height, sizeof(int *));
140 for (i = 0; i < height; i++) {
141     img[i] = (int *)calloc(width, sizeof(int));
142     bufArr[i] = (int *)calloc(width, sizeof(int));
143 }
144 if (img == NULL || bufArr == NULL) {
145     printf("failed to get enough of memory\n");
146     return -1;
147 }
148
149 for (y = 0; y < height; y++) {
150     for (x = 0; x < width; x++) {
151         fscanf(fpi, "%d", &img[y][x]);
152     }
153 } // 画像中 (x,y)の位置にある画素をimg[y][x]で参照できる。
154
155 fclose(fpi);
156
157 int dir_current;
158 int p_c = 0; // 1 黒 0 白
159 int label = 1;
160 double length, C_value;
161 int total, cpx, cpy;
162 for (y = 0; y < height; y++) {
163     for (x = 0; x < width; x++) {
164         if (img[y][x] && !bufArr[y][x] && !p_c) {
165             bufArr[y][x] = 1;
166             dir_current = 6;
167             length = followOutlineAndGetLength(img, bufArr, width, height, &
                dir_current, x, y, label);
168             fillInside(img, bufArr, width, height);
169             total = getnumTotal(bufArr, label, width, height);
170             cpx = getCPX(bufArr, label, width, height);
171             cpy = getCPY(bufArr, label, width, height);
172             C_value = calC(total, length);

```

```

173         if(C_value < 0.9f) {
174             rmlabel(bufArr, &label, width, height);
175         } else {
176             printf("label : %d, square : %d, length : %5.2f, cpoint : (%d, %d
                ), Cvalue : %5.2f\n", label, total, length, cpx, cpy, C_value);
177             label++;
178         }
179     }
180     p_c = img[y][x] ? 1 : 0;
181 }
182 }
183
184 maxvalue = 255;
185 for (y = 0; y < height; y++) {
186     for (x = 0; x < width; x++) {
187         if(bufArr[y][x] == -1) bufArr[y][x] = 0;
188         img[y][x] = bufArr[y][x] * (maxvalue / (label - 1));
189     }
190 }
191
192 // 画像データをファイル fnameout に出力する
193 fpo = fopen(fnameout, "w");
194
195 if (fpo == NULL) printf("Reading file open error ! %s\n", fnameout);
196
197 fprintf(fpo, "P2\n");
198 fprintf(fpo, "# %s\n", fnameout);
199 fprintf(fpo, "%d %d \n", width, height);
200 fprintf(fpo, "%d\n", maxvalue);
201
202 int count = 0;
203
204 for (y = 0; y < height; y++) {
205     for (x = 0; x < width; x++) {
206         fprintf(fpo, "%1d ", img[y][x]);
207         count++;
208         if (count % 18 == 0) {
209             fprintf(fpo, "\n"); // 1行 18個画素
210         }
211     }
212 }
213 fclose(fpo); // 出力画像をクローズする
214
215 for (i = 0; i < height; i++) {
216     free(img[i]);
217     free(bufArr[i]);
218 }
219 free(img);
220 free(bufArr);
221 }

```
